Lesson 68: Long Short-Term Memory and Gating Mechanisms

AI/ML Mathematics

Contents

1 Motivation and Overview **2**

2 References **3**

3 The Gating Principle **3**

4 LSTM Cell Architecture **4**
- 4.1 The Forget Gate . . . 4
- 4.2 The Input Gate and Candidate Cell State . . . 4
- 4.3 Cell State Update . . . 4
- 4.4 The Output Gate and Hidden State . . . 5

5 Complete LSTM Forward Pass **5**

6 Backpropagation Through the LSTM **6**

7 Gradient Flow Analysis **8**

8 LSTM Variants **9**
- 8.1 Peephole Connections . . . 9
- 8.2 Coupled Input-Forget Gate . . . 9

9 Exercises **10**

10 Summary **10**

11 Further Reading **11**

1 Motivation and Overview

In Lesson 67, we proved that the gradient of the loss with respect to an earlier hidden state decays exponentially as $\gamma^{t-\tau}$ when $\|\mathbf{U}\| < 1$. This means a vanilla RNN cannot learn to associate an input at time $\tau = 1$ with a target at time $t = 100$: the gradient signal is attenuated by a factor of γ^{99}, which is negligible for any $\gamma < 1$. The root cause is the *multiplicative* nature of gradient flow through the recurrence $\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b})$: each backward step multiplies by $\mathrm{diag}(1 - \mathbf{h}_t^2)\mathbf{U}$, and the product of many such matrices either shrinks or grows.

What if we could replace this multiplicative chain with an *additive* update? The Long Short-Term Memory (LSTM) architecture, introduced by Hochreiter and Schmidhuber [1997], does exactly this. It introduces a *cell state* $\mathbf{c}_t$ that is updated additively: $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$. When the forget gate $\mathbf{f}_t \approx \mathbf{1}$, gradients flow through the cell state with minimal decay, creating a "gradient highway" that allows information to persist over hundreds of time steps. The tools developed in this lesson will let us formalize the LSTM architecture, derive the BPTT algorithm through its gated structure, and prove that the cell state update mitigates the vanishing gradient problem.

Why LSTMs Matter for RL/ML

- **Recurrent policies in POMDPs.** LSTM-based policies in PPO and A3C can maintain belief states over long episodes, enabling effective decision-making under partial observability.
- **Long-range credit assignment.** The LSTM's ability to preserve gradient flow over hundreds of steps is critical for tasks where rewards are sparse or delayed.
- **Sequence modeling benchmarks.** LSTMs achieve state-of-the-art results on speech recognition, machine translation, and language modeling—tasks that require capturing long-range dependencies.
- **Gating as a design principle.** The gating mechanism introduced by LSTMs appears throughout deep RL: highway networks, residual connections, and attention mechanisms all use similar ideas of controlled information flow.

Prerequisites. Vanilla RNN architecture and BPTT (Lesson 67, Definitions 4.2–5.3 and Theorems 6.1–6.4). Vanishing gradient problem and the Jacobian product structure (Lesson 67, Section 7). Sigmoid function $\sigma(z) = 1/(1 + e^{-z})$ and its derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ (Lesson 63, Section 3). Hadamard (element-wise) product.

Learning objectives. After completing this lesson, you will be able to:

- **Define** the LSTM cell architecture, including the forget gate, input gate, candidate cell state, cell state update, and output gate.
- **Derive** the complete LSTM forward equations and **compute** the parameter count.
- **Derive** the BPTT algorithm through the LSTM, including the cell state gradient recurrence and gate gradients.
- **Prove** that the cell state gradient recurrence $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \mathrm{diag}(\mathbf{f}_t)$ allows gradients to flow with minimal decay when $\mathbf{f}_t \approx \mathbf{1}$.
- **Compare** the gradient flow in LSTMs versus vanilla RNNs and **explain** why the additive cell state update solves the vanishing gradient problem.
- **Describe** LSTM variants (peephole connections, coupled input-forget gate) and **evaluate** their trade-offs.

2 References

- Hochreiter and Schmidhuber [1997] — The original LSTM paper: architecture, gating mechanisms, and experimental validation on long-range dependency tasks.
- Gers et al. [2000] — Introduction of the forget gate, which was absent from the original LSTM and is now considered essential.
- Greff et al. [2017] — Comprehensive empirical study of LSTM variants, ablation studies, and practical guidelines.
- Goodfellow et al. [2016, Chapter 10] — Textbook treatment of LSTM architecture and its relationship to gated recurrent networks.
- Graves [2012] — Supervised sequence labelling with LSTMs, including CTC and bidirectional architectures.

3 The Gating Principle

The central idea behind the LSTM is to control information flow using *gates*: vectors whose components lie in $(0, 1)$, which act as soft switches via element-wise multiplication.

Definition 3.1 (Sigmoid Gate)**.** A *sigmoid gate* is a vector $\mathbf{g} \in (0, 1)^m$ computed as:

$$\mathbf{g} = \sigma(\mathbf{W}_g \mathbf{x}_t + \mathbf{U}_g \mathbf{h}_{t-1} + \mathbf{b}_g),$$

where $\sigma(z) = 1/(1 + e^{-z})$ is the logistic sigmoid applied element-wise.

Definition 3.2 (Hadamard Product)**.** For vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^m$, the *Hadamard product* $\mathbf{a} \odot \mathbf{b} \in \mathbb{R}^m$ is defined component-wise:

$$(\mathbf{a} \odot \mathbf{b})_j = a_j \cdot b_j, \quad j = 1, \ldots, m.$$

In matrix form: $\mathbf{a} \odot \mathbf{b} = \mathrm{diag}(\mathbf{a})\mathbf{b} = \mathrm{diag}(\mathbf{b})\mathbf{a}$.

Remark 3.3 (Gating as Soft Selection). The operation $\mathbf{g} \odot \mathbf{v}$ where $\mathbf{g} \in (0, 1)^m$ acts as a soft selection: when $g_j \approx 1$, component v_j passes through; when $g_j \approx 0$, it is suppressed. This allows the network to *learn* which information to keep, which to discard, and which to update—all in a differentiable manner.

Proposition 3.4 (Derivative Properties)**.** *The activation functions used in LSTM gates satisfy:*

$$\frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z)), \tag{1}$$

$$\frac{d \tanh}{dz} = 1 - \tanh^2(z) = \mathrm{sech}^2(z). \tag{2}$$

Both derivatives can be expressed in terms of the function values, enabling efficient computation during backpropagation.

Proof. For the sigmoid: $\frac{d}{dz}\frac{1}{1+e^{-z}} = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z)(1 - \sigma(z))$. The tanh derivative follows from $\tanh(z) = 2\sigma(2z) - 1$, giving $\frac{d \tanh}{dz} = 2 \cdot 2\sigma(2z)(1 - \sigma(2z)) = 1 - \tanh^2(z)$. □

We now construct the LSTM cell from four gating components.

4 LSTM Cell Architecture

The LSTM cell maintains two state vectors at each time step:

- The *cell state* $\mathbf{c}_t \in \mathbb{R}^m$: long-term memory, updated additively.
- The *hidden state* $\mathbf{h}_t \in (-1, 1)^m$: short-term memory and output, derived from the cell state.

Four components govern the cell update at each time step.

4.1 The Forget Gate

Definition 4.1 (Forget Gate)**.** The *forget gate* $\mathbf{f}_t \in (0, 1)^m$ at time t determines which components of the previous cell state to retain:

$$\boxed{\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f)}, \tag{3}$$

where $\mathbf{W}_f \in \mathbb{R}^{m \times n}$, $\mathbf{U}_f \in \mathbb{R}^{m \times m}$, and $\mathbf{b}_f \in \mathbb{R}^m$.

Remark 4.2 (Interpretation). When $f_t^{(j)} \approx 0$, the j-th component of the cell state is "forgotten" (erased). When $f_t^{(j)} \approx 1$, it is preserved. The forget gate was not part of the original LSTM [Hochreiter and Schmidhuber, 1997] but was added by Gers et al. [2000] and is now considered essential.

4.2 The Input Gate and Candidate Cell State

Definition 4.3 (Input Gate)**.** The *input gate* $\mathbf{i}_t \in (0, 1)^m$ controls how much of the new candidate information to incorporate:

$$\boxed{\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i)}, \tag{4}$$

where $\mathbf{W}_i \in \mathbb{R}^{m \times n}$, $\mathbf{U}_i \in \mathbb{R}^{m \times m}$, and $\mathbf{b}_i \in \mathbb{R}^m$.

Definition 4.4 (Candidate Cell State)**.** The *candidate cell state* $\tilde{\mathbf{c}}_t \in (-1, 1)^m$ represents potential new information:

$$\boxed{\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c)}, \tag{5}$$

where $\mathbf{W}_c \in \mathbb{R}^{m \times n}$, $\mathbf{U}_c \in \mathbb{R}^{m \times m}$, and $\mathbf{b}_c \in \mathbb{R}^m$.

4.3 Cell State Update

Definition 4.5 (Cell State Update)**.** The cell state is updated by combining retained old information with gated new information:

$$\boxed{\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t}. \tag{6}$$

Remark 4.6 (Additive Structure). The cell state update (6) is *additive*: $\mathbf{c}_t$ is a weighted sum of the previous cell state and a new candidate. This is the key architectural innovation. Unlike the vanilla RNN's multiplicative update $\mathbf{h}_t = \tanh(\mathbf{U}\mathbf{h}_{t-1} + \cdots)$, the additive structure allows the Jacobian $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1}$ to be close to the identity, enabling gradient flow over long distances.

4.4 The Output Gate and Hidden State

Definition 4.7 (Output Gate)**.** The *output gate* $\mathbf{o}_t \in (0,1)^m$ determines which components of the cell state are exposed as the hidden state:

$$\boxed{\mathbf{o}_t = \sigma(\mathbf{W}_o\mathbf{x}_t + \mathbf{U}_o\mathbf{h}_{t-1} + \mathbf{b}_o)}, \tag{7}$$

where $\mathbf{W}_o \in \mathbb{R}^{m\times n}$, $\mathbf{U}_o \in \mathbb{R}^{m\times m}$, and $\mathbf{b}_o \in \mathbb{R}^m$.

Definition 4.8 (Hidden State Output)**.** The hidden state is computed by filtering the cell state through the output gate:

$$\boxed{\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)}. \tag{8}$$

The tanh squashes the cell state to $(-1,1)$, and the output gate selects which components to expose.

Example 4.9 (LSTM as Selective Memory)**.** Consider a language model processing the sentence "The cat, which was sitting on the mat, **was** sleeping." The LSTM must remember that the subject is singular ("cat") across the relative clause to correctly predict the verb "was" (not "were"):

- At "cat": the input gate stores subject number information in a cell state component.
- During the relative clause: the forget gate keeps this component close to 1, preserving the information.
- At the verb position: the output gate exposes the number information to influence the prediction.

A vanilla RNN would lose this information due to the vanishing gradient over the 7-word gap.

5 Complete LSTM Forward Pass

LSTM Forward Equations

At each time step $t = 1, \ldots, T$:

$$\begin{aligned}
\mathbf{f}_t &= \sigma(\mathbf{W}_f\mathbf{x}_t + \mathbf{U}_f\mathbf{h}_{t-1} + \mathbf{b}_f), && \text{[Forget Gate]} && (9)\\
\mathbf{i}_t &= \sigma(\mathbf{W}_i\mathbf{x}_t + \mathbf{U}_i\mathbf{h}_{t-1} + \mathbf{b}_i), && \text{[Input Gate]} && (10)\\
\tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c\mathbf{x}_t + \mathbf{U}_c\mathbf{h}_{t-1} + \mathbf{b}_c), && \text{[Candidate]} && (11)\\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, && \text{[Cell Update]} && (12)\\
\mathbf{o}_t &= \sigma(\mathbf{W}_o\mathbf{x}_t + \mathbf{U}_o\mathbf{h}_{t-1} + \mathbf{b}_o), && \text{[Output Gate]} && (13)\\
\mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t), && \text{[Hidden State]} && (14)
\end{aligned}$$

with initial conditions $\mathbf{h}_0 = \mathbf{0} \in \mathbb{R}^m$ and $\mathbf{c}_0 = \mathbf{0} \in \mathbb{R}^m$.

Proposition 5.1 (LSTM Parameter Count)**.** *For an LSTM with input dimension n and hidden dimension m, the total number of trainable parameters (excluding the output layer) is:*

$$P = 4(mn + m^2 + m) = 4m(n + m + 1). \tag{15}$$

Proof. Each of the four components (forget gate, input gate, candidate, output gate) has:

- Input weight matrix: $m \times n$ parameters,
- Recurrent weight matrix: $m \times m$ parameters,
- Bias vector: m parameters.

Total: $4 \times (mn + m^2 + m) = 4m(n + m + 1)$. □

Remark 5.2 (LSTM vs Vanilla RNN Parameters). An LSTM has approximately 4× more parameters than a vanilla RNN of the same hidden dimension, due to the four separate gate/candidate components. Despite this overhead, the improved gradient flow typically justifies the additional cost.

Definition 5.3 (Compact Matrix Formulation)**.** For computational efficiency, define concatenated parameters:

$$\mathbf{W} = \begin{pmatrix} \mathbf{W}_f \\ \mathbf{W}_i \\ \mathbf{W}_c \\ \mathbf{W}_o \end{pmatrix} \in \mathbb{R}^{4m\times n}, \quad \mathbf{U} = \begin{pmatrix} \mathbf{U}_f \\ \mathbf{U}_i \\ \mathbf{U}_c \\ \mathbf{U}_o \end{pmatrix} \in \mathbb{R}^{4m\times m}, \quad \mathbf{b} = \begin{pmatrix} \mathbf{b}_f \\ \mathbf{b}_i \\ \mathbf{b}_c \\ \mathbf{b}_o \end{pmatrix} \in \mathbb{R}^{4m}. \tag{16}$$

All pre-activations are computed in a single operation:

$$\mathbf{z}_t = \mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b} \in \mathbb{R}^{4m},$$

then split into four segments of size m and passed through the appropriate activations (σ for gates, tanh for candidate). This reduces the number of matrix multiplications from 8 to 2.

Example 5.4 (LSTM Parameter Count)**.** For an LSTM processing word embeddings of dimension $n = 300$ with hidden size $m = 512$:

$$P = 4 \times 512 \times (300 + 512 + 1) = 4 \times 512 \times 813 = 1{,}664{,}256 \approx 1.66\text{M parameters.}$$

A vanilla RNN of the same dimensions has $512 \times (300 + 512 + 1) = 416{,}256 \approx 0.42$M, about 4× fewer.

6 Backpropagation Through the LSTM

Given a sequence loss $\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t$, we derive the gradients by propagating errors backward through the LSTM computational graph. Define the error signals:

$$\boldsymbol{\delta}_t^h = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t}, \tag{17}$$

$$\boldsymbol{\delta}_t^c = \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t}. \tag{18}$$

Theorem 6.1 (Cell State Gradient Recurrence)**.** *The cell state error signal satisfies the recurrence:*

$$\boxed{\boldsymbol{\delta}_t^c = \boldsymbol{\delta}_t^h \odot \mathbf{o}_t \odot (1 - \tanh^2(\mathbf{c}_t)) + \boldsymbol{\delta}_{t+1}^c \odot \mathbf{f}_{t+1}}. \tag{19}$$

Proof. The cell state $\mathbf{c}_t$ influences the total loss through two paths:

1. Through the current hidden state: $\mathbf{c}_t \to \tanh(\mathbf{c}_t) \to \mathbf{h}_t$.
2. Through the next cell state: $\mathbf{c}_t \to \mathbf{c}_{t+1}$ via the cell update equation.

For path (1): since $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$,

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{c}_t} = \operatorname{diag}(\mathbf{o}_t) \operatorname{diag}(1 - \tanh^2(\mathbf{c}_t)).$$

So the contribution is $\boldsymbol{\delta}_t^h \odot \mathbf{o}_t \odot (1 - \tanh^2(\mathbf{c}_t))$.

For path (2): since $\mathbf{c}_{t+1} = \mathbf{f}_{t+1} \odot \mathbf{c}_t + \mathbf{i}_{t+1} \odot \tilde{\mathbf{c}}_{t+1}$, and in the standard LSTM (without peephole connections) the gates $\mathbf{f}_{t+1}$, $\mathbf{i}_{t+1}$, and $\tilde{\mathbf{c}}_{t+1}$ do not depend on $\mathbf{c}_t$:

$$\frac{\partial \mathbf{c}_{t+1}}{\partial \mathbf{c}_t} = \operatorname{diag}(\mathbf{f}_{t+1}).$$

Combining both paths gives the recurrence (19). □

LSTM Gradient Highway

The cell state gradient recurrence $\boldsymbol{\delta}_t^c = (\text{local term}) + \boldsymbol{\delta}_{t+1}^c \odot \mathbf{f}_{t+1}$ is the key to the LSTM's success. The gradient flows backward through the cell state by multiplication with the forget gate values $\mathbf{f}_{t+1}$. When $\mathbf{f}_t \approx \mathbf{1}$:

$$\boldsymbol{\delta}_1^c \approx \boldsymbol{\delta}_T^c \cdot \prod_{t=2}^{T} \operatorname{diag}(\mathbf{f}_t) \approx \boldsymbol{\delta}_T^c,$$

achieving near-perfect gradient preservation over T steps.

The gate pre-activation gradients are used to compute weight gradients. Define the pre-activation error signals:

$$\boldsymbol{\delta}_t^f = \boldsymbol{\delta}_t^c \odot \mathbf{c}_{t-1} \odot \mathbf{f}_t \odot (1 - \mathbf{f}_t), \qquad \text{[Forget Gate]} \tag{20}$$

$$\boldsymbol{\delta}_t^i = \boldsymbol{\delta}_t^c \odot \tilde{\mathbf{c}}_t \odot \mathbf{i}_t \odot (1 - \mathbf{i}_t), \qquad \text{[Input Gate]} \tag{21}$$

$$\boldsymbol{\delta}_t^{\tilde{c}} = \boldsymbol{\delta}_t^c \odot \mathbf{i}_t \odot (1 - \tilde{\mathbf{c}}_t^2), \qquad \text{[Candidate]} \tag{22}$$

$$\boldsymbol{\delta}_t^o = \boldsymbol{\delta}_t^h \odot \tanh(\mathbf{c}_t) \odot \mathbf{o}_t \odot (1 - \mathbf{o}_t). \qquad \text{[Output Gate]} \tag{23}$$

Theorem 6.2 (LSTM Weight Gradients). *The gradients with respect to the forget gate parameters are:*

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_f} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^f \mathbf{x}_t^\top, \tag{24}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{U}_f} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^f \mathbf{h}_{t-1}^\top, \tag{25}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_f} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^f. \tag{26}$$

Analogous expressions hold for the input gate, candidate, and output gate parameters (replacing the superscript f with i, $\tilde{c}$, and o respectively).

Proof. Consider the forget gate at time t. Let $\mathbf{z}_t^f = \mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f$ be the pre-activation. Then $\mathbf{f}_t = \sigma(\mathbf{z}_t^f)$ and the chain rule gives:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_t^f} = \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t} \odot \frac{\partial \mathbf{c}_t}{\partial \mathbf{f}_t} \odot \frac{\partial \mathbf{f}_t}{\partial \mathbf{z}_t^f} = \boldsymbol{\delta}_t^c \odot \mathbf{c}_{t-1} \odot \mathbf{f}_t \odot (1 - \mathbf{f}_t) = \boldsymbol{\delta}_t^f.$$

Since $\mathbf{z}_t^f = \mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f$, the weight gradients follow by the standard matrix calculus: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_f} = \sum_t \boldsymbol{\delta}_t^f \mathbf{x}_t^\top$. The derivations for the other gates are identical in structure. □

Remark 6.3 (Hidden State Error Signal). The hidden state error $\boldsymbol{\delta}_t^h$ receives contributions from the current loss $\mathcal{L}_t$ and from the backpropagation through the gates at time $t+1$ (since the gates at $t+1$ depend on $\mathbf{h}_t$). Specifically:

$$\boldsymbol{\delta}_t^h = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} + \sum_{g \in \{f,i,c,o\}} \mathbf{U}_g^\top \boldsymbol{\delta}_{t+1}^g, \tag{27}$$

where the sum is over the four gate/candidate pre-activation errors at time $t+1$.

7 Gradient Flow Analysis

We now formally compare the gradient flow properties of the LSTM with those of the vanilla RNN.

Theorem 7.1 (LSTM Cell State Jacobian)**.** *In the standard LSTM (without peephole connections), the Jacobian of* $\mathbf{c}_t$ *with respect to* $\mathbf{c}_{t-1}$ *is:*

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \operatorname{diag}(\mathbf{f}_t). \tag{28}$$

Proof. Taking the partial derivative of the cell state update $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$ with respect to $\mathbf{c}_{t-1}$:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \operatorname{diag}(\mathbf{f}_t) + \mathbf{c}_{t-1} \odot \frac{\partial \mathbf{f}_t}{\partial \mathbf{c}_{t-1}} + \tilde{\mathbf{c}}_t \odot \frac{\partial \mathbf{i}_t}{\partial \mathbf{c}_{t-1}} + \mathbf{i}_t \odot \frac{\partial \tilde{\mathbf{c}}_t}{\partial \mathbf{c}_{t-1}}.$$

In the standard LSTM, the gates depend on $\mathbf{x}_t$ and $\mathbf{h}_{t-1}$ but *not directly* on $\mathbf{c}_{t-1}$, so the last three terms vanish: $\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \operatorname{diag}(\mathbf{f}_t)$. □

Corollary 7.2 (Long-Range Cell State Gradient)**.** *The gradient of* $\mathcal{L}$ *with respect to an earlier cell state* $\mathbf{c}_\tau$ *(*$\tau < t$*) through the cell state pathway is:*

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_\tau} = \prod_{i=\tau+1}^{t} \operatorname{diag}(\mathbf{f}_i). \tag{29}$$

Proof. By Theorem 7.1 and the chain rule: $\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_\tau} = \prod_{i=\tau+1}^{t} \frac{\partial \mathbf{c}_i}{\partial \mathbf{c}_{i-1}} = \prod_{i=\tau+1}^{t} \operatorname{diag}(\mathbf{f}_i)$. □

Theorem 7.3 (Vanishing Gradient Mitigation)**.** *When* $\mathbf{f}_t \approx \mathbf{1}$ *for all* t*, the cell state gradient satisfies:*

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_\tau} \approx \mathbf{I}, \tag{30}$$

preventing the exponential decay characteristic of vanilla RNNs. More precisely, for component j*:*

$$\left[\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_\tau}\right]_{jj} = \prod_{i=\tau+1}^{t} f_i^{(j)}.$$

If $f_i^{(j)} \geq 1 - \epsilon$ *for all* i*, then* $\prod_{i=\tau+1}^{t} f_i^{(j)} \geq (1-\epsilon)^{t-\tau} \geq 1 - (t-\tau)\epsilon$ *for small* ϵ*.*

Proof. When $\mathbf{f}_i \approx \mathbf{1}$ for all i, each diagonal matrix $\operatorname{diag}(\mathbf{f}_i) \approx \mathbf{I}$. The product of $t - \tau$ such matrices is approximately $\mathbf{I}$. For the precise bound, use $(1-\epsilon)^k \geq 1 - k\epsilon$ (valid for $0 \leq \epsilon \leq 1$ and $k \geq 1$, by Bernoulli's inequality). □

Example 7.4 (LSTM vs Vanilla RNN Gradient Flow)**.** Consider a sequence of length $T = 100$ with time gap $k = 99$.

- **Vanilla RNN** with $\|\mathbf{U}\| = 0.9$: gradient decays by $0.9^{99} \approx 3 \times 10^{-5}$ (effectively zero).
- **LSTM** with $f_i^{(j)} = 0.99$ for all i: gradient decays by $0.99^{99} \approx 0.37$ (substantial signal preserved).
- **LSTM** with $f_i^{(j)} = 1.0$: gradient decays by $1.0^{99} = 1$ (perfect preservation).

The LSTM's forget gate can maintain gradient flow over arbitrarily long sequences by keeping $\mathbf{f}_t$ close to $\mathbf{1}$.

Forget Gate Bias Initialization

In practice, the forget gate bias $\mathbf{b}_f$ is often initialized to a positive value (e.g., 1 or 2) so that $\sigma(\mathbf{b}_f) \approx 1$ at the start of training. This ensures that the LSTM begins with near-perfect gradient flow through the cell state, allowing it to learn long-range dependencies from the outset. This trick, recommended by Gers et al. [2000], is widely used in deep RL implementations.

Gradient Flow Through Hidden State

The analysis above concerns gradient flow through the *cell state* pathway. Gradients also flow through the *hidden state* pathway via the gate recurrences (Equation 27), which involves multiplication by the recurrent weight matrices $\mathbf{U}_g^\top$. This pathway can still suffer from vanishing gradients, but the cell state highway provides an alternative route that bypasses this bottleneck.

8 LSTM Variants

8.1 Peephole Connections

Definition 8.1 (Peephole LSTM)**.** *Peephole connections*, introduced by Gers et al. [2000], allow the gates to access the cell state directly:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f\mathbf{x}_t + \mathbf{U}_f\mathbf{h}_{t-1} + \mathbf{V}_f \odot \mathbf{c}_{t-1} + \mathbf{b}_f), \tag{31}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i\mathbf{x}_t + \mathbf{U}_i\mathbf{h}_{t-1} + \mathbf{V}_i \odot \mathbf{c}_{t-1} + \mathbf{b}_i), \tag{32}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o\mathbf{x}_t + \mathbf{U}_o\mathbf{h}_{t-1} + \mathbf{V}_o \odot \mathbf{c}_t + \mathbf{b}_o), \tag{33}$$

where $\mathbf{V}_f, \mathbf{V}_i, \mathbf{V}_o \in \mathbb{R}^m$ are diagonal peephole weight vectors.

Remark 8.2 (Peephole Trade-offs). Peephole connections add $3m$ parameters and make the Jacobian $\partial\mathbf{c}_t/\partial\mathbf{c}_{t-1}$ more complex (no longer purely $\mathrm{diag}(\mathbf{f}_t)$). Empirical studies [Greff et al., 2017] find that peephole connections provide marginal benefit on most tasks and are not commonly used in modern implementations.

8.2 Coupled Input-Forget Gate

Definition 8.3 (Coupled Gate LSTM)**.** The *coupled input-forget gate* variant enforces the constraint $\mathbf{i}_t = \mathbf{1} - \mathbf{f}_t$, reducing the cell state update to:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + (\mathbf{1} - \mathbf{f}_t) \odot \tilde{\mathbf{c}}_t. \tag{34}$$

Remark 8.4 (Coupled Gate Interpretation). The coupled gate makes an explicit trade-off: the network can either retain old information ($f_t^{(j)} \approx 1$) or write new information ($f_t^{(j)} \approx 0$), but not both simultaneously. This reduces the parameter count by $m(n + m + 1)$ (one fewer gate) while maintaining the core functionality.

Example 8.5 (Coupled Gate vs Standard LSTM)**.** With the coupled gate, the cell state update $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + (1 - \mathbf{f}_t) \odot \tilde{\mathbf{c}}_t$ is a convex combination of the old cell state and the candidate. This ensures that the cell state norm does not grow unboundedly (since $|\tilde{c}_t^{(j)}| < 1$), providing implicit regularization. In the standard LSTM, $\mathbf{c}_t$ can grow without bound if both $\mathbf{f}_t$ and $\mathbf{i}_t$ are simultaneously large.

9 Exercises

Exercise 9.1 (LSTM Forward Pass)**.** $(*)$
Consider a scalar LSTM ($n = m = 1$) with $W_f = W_i = W_c = W_o = 1, U_f = U_i = U_c = U_o = 0.5$, $b_f = 1$, and $b_i = b_c = b_o = 0$. Starting from $h_0 = c_0 = 0$, compute the cell state c_1, c_2 and hidden state h_1, h_2 for inputs $x_1 = 1, x_2 = -1$.

Exercise 9.2 (Cell State Gradient)**.** $(**)$
For an LSTM with $m = 2$ and constant forget gate values $\mathbf{f}_t = (0.95, 0.99)^\top$ for all t, compute the cell state gradient $\partial \mathbf{c}_{100}/\partial \mathbf{c}_1$ and compare it with the gradient in a vanilla RNN with $\|\mathbf{U}\| = 0.95$. Which architecture preserves the gradient signal better over 100 steps?

Exercise 9.3 (LSTM vs Coupled Gate Parameter Count)**.** $(*)$
For an LSTM with input dimension $n = 256$ and hidden dimension $m = 512$, compute the parameter count for: (a) the standard LSTM, (b) the coupled input-forget gate variant. What percentage of parameters does the coupled variant save?

Exercise 9.4 (Peephole Jacobian)**.** $(***)$
Derive the Jacobian $\partial \mathbf{c}_t/\partial \mathbf{c}_{t-1}$ for the peephole LSTM (Definition 8.1). Show that it is no longer $\mathrm{diag}(\mathbf{f}_t)$ and identify the additional terms. Under what conditions do these terms improve or degrade gradient flow compared to the standard LSTM?

Hint: The forget and input gates now depend on $\mathbf{c}_{t-1}$ through the peephole connections.

Exercise 9.5 (Gradient Flow with Saturated Gates)**.** $(**)$
Suppose the forget gate saturates at $\mathbf{f}_t = \mathbf{1}$ (all components exactly 1). Show that the cell state gradient $\partial \mathbf{c}_t/\partial \mathbf{c}_\tau = \mathbf{I}$ for all $\tau < t$. Then explain why this "perfect" gradient flow is problematic: what happens to the cell state values $\|\mathbf{c}_t\|$ as t grows, and how does this affect the numerical stability of the network?

10 Summary

- The LSTM introduces a *cell state* $\mathbf{c}_t$ updated additively: $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$. This additive structure is the key architectural innovation.
- Four gating components—forget gate $\mathbf{f}_t$, input gate $\mathbf{i}_t$, candidate $\tilde{\mathbf{c}}_t$, and output gate $\mathbf{o}_t$—control information flow using sigmoid-gated element-wise products.
- The cell state Jacobian is $\partial \mathbf{c}_t/\partial \mathbf{c}_{t-1} = \mathrm{diag}(\mathbf{f}_t)$, which is close to the identity when $\mathbf{f}_t \approx \mathbf{1}$, creating a "gradient highway" that mitigates the vanishing gradient problem.
- BPTT through the LSTM involves the cell state gradient recurrence $\boldsymbol{\delta}_t^c = \boldsymbol{\delta}_t^h \odot \mathbf{o}_t \odot (1 - \tanh^2(\mathbf{c}_t)) + \boldsymbol{\delta}_{t+1}^c \odot \mathbf{f}_{t+1}$ and four gate pre-activation errors.
- The LSTM has $4m(n+m+1)$ parameters—approximately $4\times$ more than a vanilla RNN—due to the four separate gate/candidate components.
- Variants include peephole connections (gates access cell state directly) and the coupled input-forget gate ($\mathbf{i}_t = \mathbf{1} - \mathbf{f}_t$), which trades flexibility for fewer parameters.
- Initializing the forget gate bias to a positive value ($b_f \approx 1$–2) ensures good gradient flow from the start of training.

11 Further Reading

- Lesson 69 introduces the Gated Recurrent Unit (GRU), a simplified gated architecture with fewer parameters, and covers bidirectional RNNs, deep stacked RNNs, and the encoder-decoder framework.
- Greff et al. [2017] provides a comprehensive empirical evaluation of LSTM variants (peephole, coupled gates, no-input-gate, no-forget-gate, etc.), finding that the forget gate and output activation are the most critical components.
- Hochreiter and Schmidhuber [1997] is the original LSTM paper and contains the detailed analysis of the *constant error carousel*—the precursor to the modern cell state update.
- Graves [2012] covers practical applications of LSTMs to sequence labelling, including speech recognition and handwriting generation.
- The connection between LSTMs and residual networks (both use additive shortcuts to improve gradient flow) is explored in the deep RL literature on recurrent policies (Phase 04, Lessons 77–80).

References

Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins. Learning to forget: Continual prediction with LSTM. *Neural Computation*, 12(10):2451–2471, 2000.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

Alex Graves. *Supervised Sequence Labelling with Recurrent Neural Networks*. Springer, 2012.

Klaus Greff, Rupesh Kumar Srivastava, Jan Koutník, Bas R. Steunebrink, and Jürgen Schmidhuber. LSTM: A search space odyssey. *IEEE Transactions on Neural Networks and Learning Systems*, 28(10):2222–2232, 2017.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9 (8):1735–1780, 1997.